

Documentación de Servicios: Integración del Frontend, Procesos y Negocio de la API

Esta documentación describe en detalle la capa de servicios del sistema (`src/api/services/`), la cual encapsula el 100% de la lógica de negocio dura, los motores de cálculo matemático y financiero del contrato, y las integraciones con servicios cognitivos de Inteligencia Artificial (LLM & OCR).

[!NOTE]

Regla de Arquitectura (Thin Controllers & Fat Services):

Los controladores son ligeros y solo gestionan la petición y respuesta HTTP. Toda la lógica de validación, agregación y cálculo reside estrictamente en esta capa de servicios. Un servicio no conoce los objetos `Request` o `Response` de Express, lo que permite que sus métodos sean 100% reutilizables y testeables.

Sección 1: Documentación para el Frontend (Fórmulas y Reglas de Cálculo)

Aunque el frontend consume controladores, las reglas que determinan los números y reportes que visualiza el usuario final se ejecutan en los servicios. Aquí se describen las lógicas de cálculo clave:

1. Sistema de Redondeo de Jornadas (Largest Remainder Method)

Cuando los operadores reportan subjornadas o turnos de trabajo fraccionados, el sistema suma los tiempos. Para asegurar que la suma final en la trazabilidad por recurso sea siempre exactamente `1.00`, los servicios de reportes ejecutan el método `distributeRoundedValues` con la siguiente prioridad de negocio:

- **Prioridad 3 (Más Alta):** Estado de Operación (`J`).
- **Prioridad 2 (Media):** Estado de Disponibilidad (`D`).
- **Prioridad 1 (Baja):** Estado de Mantenimiento (`MTT0`) y Fuera del Sistema/Servicio (`FS`).

- *Efecto:* Las desviaciones por decimales se ajustan primero en las jornadas productivas en operación.

2. Prioridad de Asignación de Instalaciones en Trazabilidad

Para imputar costes, el generador de trazabilidad calcula el campo `installation` del registro final aplicando el siguiente orden estricto de precedencia:

1. **Instalación Destino de la Actividad (`ActivityModel.targetInstallationActivie`):**
Modificada puntualmente por el usuario para una tarea en particular.
2. **Instalación del TEB (`BaseTebModel.placeServiceProvided`):** Dirección contractual del presupuesto asignado.
3. **Fallback por Defecto ('X'):** Si no se encuentra ninguna de las anteriores.

3. Cálculos de Importes Monetarios

El importe total (`amount`) de cada `laborDay` de trazabilidad depende de las banderas operativas del recurso:

- **Si es Mantenimiento (`isMaintenance = true`):** El importe es siempre `0.00` (no genera costo contractual).
- **Si es Fuera del Sistema (`isOutside = true`):** Se aplica un descuento penalizado del **75%** y el valor se acumula como un saldo negativo:
$$\text{Importe} = \text{Volumen} \times \text{Precio Unitario} \times -0.75$$
- **Operación Normal:**
$$\text{Importe} = \text{Volumen} \times \text{Precio Unitario}$$

Sección 2: Guía de Procesos y Negocio por Carpeta de Servicio

A continuación se detallan los 15 subdirectorios de la capa de servicios.

1. `ActivityServices (activity.service.ts)`

- **Relevancia de Negocio:** Encargado de la validación del tiempo y horas de operación declaradas en las actividades.

- **Proceso Interno:** Procesa los strings de horario "HH:mm" , calcula las horas transcurridas en minutos utilizando `parseTimeToMinutes` y verifica que la suma total declarada de actividades del día coincida con las fracciones del reporte.
-

2. BaseTebbsAndTraceability (`BaseTebbsAndTraceability.service.ts` / `trazabilityService.service.ts`)

- **Relevancia de Negocio:** El motor de integración contable del contrato. Cruza las partidas presupuestales operadas con el catálogo de TEBs/CABs/CAXs.
 - **Procesos Clave:**
 - `generateTraceability` : Consume `getDepartureLimit` de partidas para obtener el aplanado de `laborDays`. Realiza un cruce de IDs en lote contra `BaseTebbsModel` . Deduplica y acumula volúmenes agrupando por la clave compuesta `día | equipo | partida | Outside/Inside | activityId` (evitando fugas decimales).
 - `generateAndStoreInTemp` : Ejecuta la generación y la guarda en la colección temporal `TrazabilityTempModel` .
 - `commitTemp` : Migra de forma atómica los registros temporales. Los que contienen `requestNo` real se persisten en `TrazabilityHistoryModel` y los vacíos/simulados se desvían a `TraceabilityProrratioModel` .
 - `bulkCreateCatalogList` : Carga masiva mediante `bulkWrite` de Mongoose en lotes (chunks) de 500 registros.
-

3. CatalogConceptServices (`CatalogTypeDocument.service.ts`)

- **Relevancia de Negocio:** Provee los métodos para gestionar los tipos de documentos, asignando las etiquetas visuales (labels) y validando la integridad del catálogo de conceptos financieros.
-

4. CatalogEquipmentServices (`CatalogEquipment.service.ts`)

- **Relevancia de Negocio:** Gobierna la asignación física de grúas, tractores, moduladores y cuadrillas de cabos/maniobristas.
 - **Procesos Clave:**
 - `assignCaboAndManeuverer` : Vincula maniobristas a cabos líderes.
 - `validateEquipmentToShip` : Valida si el equipo solicitado cumple con las restricciones de operación sobre barcos específicos del contrato.
-

5. `ConfigServices` (`configService.service.ts`)

- **Relevancia de Negocio:** Resuelve en caché o memoria los listados de catálogos paramétricos.
 - **Proceso Interno:** Ejecuta búsquedas masivas indexadas por `key` (`ship`, `installationDestination`) y las expone en diccionarios Map convirtiendo `ObjectIds` a string (`String(id)`) para evitar el cuello de botella del patrón de consultas N+1 en bases de datos NoSQL.
-

6. `ConnectionServices` (`connection.service.ts`)

- **Relevancia de Negocio:** Mantiene el control del estado y los identificadores de sockets conectados de los escáneres Electron en la base de datos de auditoría (`ConnectionModel`).
-

7. `DashboardSerivces` (`dashboard.service.ts` / `dashboardMetric.service.ts`)

- **Relevancia de Negocio:** Consolida la información financiera y operacional de todos los históricos de trazabilidad.
 - **Proceso Interno:** Realiza agregaciones complejas (`$group` , `$match` , `$project`) en MongoDB sobre las colecciones de historial y prorrateo para retornar sumas de costo productivo por categorías, dispersión de gastos, y comparar la facturación acumulada contra la meta establecida.
-

8. `DepartureServices` (`departure.service.ts`)

- **Relevancia de Negocio:** El servicio de mayor volumen de código. Administra las partidas presupuestales contractuales y los equipos asignados a ellas (`SubItemsModel`).
 - **Procesos Clave:**
 - `getDepartureLimit` : Itera y recopila los `laborDays` e items laborados en un rango de fechas. Si hay un desglose de actividades (`activitiesBreakdown`), realiza el mapeo de instalaciones y embarcaciones prioritarias.
 - `recalculateAllDepartures` : Recalcula de forma masiva los `laborDays` de todas las partidas en un rango de fechas.
 - **Kilometraje (Fletes):** Evalúa de forma no-acumulativa si el kilometraje cae en los rangos directos de cada partida y redirige kilómetros excedentes.
-

9. `EstimatedCostServices` (`estimatedCost.service.ts`)

- **Relevancia de Negocio:** Calcula los costos estimados agregados de las partidas de forma mensual para proveer un balance financiero preliminar previo a la ejecución de la trazabilidad.
-

10. `FictitiousTebServices` (`fictitiousTeb.service.ts`)

- **Relevancia de Negocio:** Provee la lógica para inyectar TEBs provisionales autogenerados cuando los reportes operativos hacen match con solicitudes inexistentes, previniendo el bloqueo del motor de trazabilidad contable.
-

11. `JornDayServices` (`jornDay.service.ts`)

- **Relevancia de Negocio:** Gestión de jornadas horarias operativas y administración del flag de absorción de redondeos decimales residuales (`isAdjustment`).
-

12. `ReportServices` (`report.service.ts` / `tempDailyReports.service.ts`)

- **Relevancia de Negocio:** Administra el ciclo de vida de los reportes diarios de trabajo físico y de cuadrillas.
- **Procesos Clave:**

- `createReport` / `updateReport` : Persiste la cabecera, crea las actividades relacionadas y actualiza de inmediato el `laborDay` operativo del equipo llamando a `recalculateResourceDailyState`.
 - `validateVolumeForResource` : Valida que la suma fraccionaria de las actividades de un equipo o recurso no exceda la jornada límite diaria de `1.0`.
-

13. `ResourceServices` (`resource.service.ts`)

- **Relevancia de Negocio:** Controla el catálogo de recursos adicionales y materiales operados en el contrato.
-

14. `ScannerServices` (`azure-ocr.service.ts` / `gemini-daily-report-service.ts` / `azure-ocr-pool.service.ts`)

- **Relevancia de Negocio:** Motor cognitivo encargado del procesamiento digital de reportes PDF cargados físicamente.
 - **Proceso Interno:**
 1. `azure-ocr-pool.service.ts` administra la cola de peticiones concurrentes para evitar errores de tasa límite (HTTP 429) con las llamadas paralelas a Azure Form Recognizer.
 2. Azure extrae los textos y tablas del reporte.
 3. `gemini-daily-report-service.ts` (Gemini Pro) interpreta la proza cruda extraída de Azure, realiza similitudes fonéticas con el catálogo maestro de equipos de MongoDB y estructura el JSON del reporte diario para revisión del usuario.
-

15. `iaAnalistaServices` (`claude.service.ts` / `validator.service.ts` / `transformer.service.ts` / `export.service.ts`)

- **Relevancia de Negocio:** Módulo de IA estratégica multi-agente que audita costos.
- **Proceso Interno:**
 1. `transformer.service.ts` (ETL) recopila datos del dashboard y los transforma en diccionarios legibles para el LLM.

2. `claude.service.ts` (Nicolás Style) corre una arquitectura Map-Reduce: analiza de forma individual los KPI operados (Map) y los resume en un informe de 8 puntos ejecutivos (Reduce).
3. `validator.service.ts` (Auditor Gemini) ejecuta un análisis matemático silencioso cruzando las afirmaciones y cifras redactadas por Claude contra los datos reales de Mongoose. Gemini corrige directamente las cifras inconsistentes en el texto original antes de entregarlo.
4. `export.service.ts` genera las versiones en DOCX, PDF y PPTX sanitizando el contenido.